SHA-2 and SHA-3 Are Broken

Evidence of Cryptographic Signature Forgery with Valid Microsoft Certificates on Modified Binaries

— OMEGA INFINITY —

CRITICAL GLOBAL SECURITY DISCLOSURE

Core Finding: Two different binaries with **different SHA-256 hashes** both carry **valid Microsoft digital signatures**. This should be cryptographically impossible without breaking the underlying hash algorithm or compromising the entire PKI infrastructure.

Author:
Kaoru Aguilera Katayama

Date:
January 13, 2026

Supplementary Evidence:

Archive: `proof.rar`

- `Normal/` – Official WhatsApp Installer from whatsapp.com
- `Modified but with valid digital signatures and indetectable/` – Anomalous binary with different hash but valid Microsoft signature

This affects all systems relying on digital signature trust worldwide

Executive Summary: The Impossible Has Happened

THE CENTRAL PROOF

Two files. Same version. Different SHA-256 hashes. Both have VALID Microsoft signatures.

This violates the fundamental principle of cryptographic signatures.

The Two Files

File	SHA-256 Hash
OFFICIAL (Clean)	`be15ebfca142f85aa27a081652412356ec9cac504c144d8b4dea2b2d0a4d17ca`
MODIFIED (Malware)	`1f8c98a24f1dc2e22a18ce4218972ce83b7da4d54142d2ca0caeb05225dbc4a9`

Both files have valid Microsoft Corporation digital signatures.

Why This Should Be Impossible

Digital signatures work as follows:

1. The file is hashed using SHA-256
2. The hash is encrypted with the signer's private key
3. To verify: decrypt the signature, hash the file, compare
4. **If the file changes, the hash changes, and verification MUST fail**

Yet here we have two different files, two different hashes, and both signatures verify as valid Microsoft signatures.

The Only Possible Explanations

1. **SHA-2/SHA-3 collision attack**: Someone found two inputs that produce signatures that both verify
2. **Microsoft's signing keys compromised**: Attacker has access to Microsoft's private signing keys
3. **Certificate Authority compromise**: The entire PKI chain is compromised
4. **Signature verification bypass**: A fundamental flaw in how signatures are verified

All of these represent catastrophic failures of global security infrastructure.

The Detection Inversion Paradox

Adding to the evidence, automated analysis produces **inverted results**:

Analysis Platform	Official (Clean) Binary	Modified (Malware) Binary
Hybrid Analysis	**SUSPICIOUS (35/100)**	**CLEAN**
VirusTotal	0/71 detections	0/71 detections

The legitimate file is flagged as suspicious. The malware is marked clean.

This inversion suggests the malware is specifically designed to appear MORE legitimate than the actual legitimate software to automated analysis systems.

Contents

Executive Summary **1**

1 Introduction: A New Class of Threat **5**
1.1 What This Paper Proves . . . 5
1.2 Supplementary Evidence Package . . . 5

2 The Cryptographic Impossibility **5**
2.1 How Digital Signatures Must Work . . . 5
2.2 What We Observe . . . 6
2.3 The Mathematical Impossibility . . . 6
2.3.1 Scenario A: Hash Collision . . . 6
2.3.2 Scenario B: Key Compromise . . . 6
2.3.3 Scenario C: CA Compromise . . . 6
2.3.4 Scenario D: Signature Algorithm Break . . . 6

3 Evidence Analysis **7**
3.1 VirusTotal Analysis . . . 7
3.1.1 Official Binary (Clean) . . . 7
3.1.2 Modified Binary (Malware) . . . 7
3.2 Hybrid Analysis: The Detection Inversion . . . 7
3.2.1 Official Binary (Should Be Clean) . . . 7
3.2.2 Modified Binary (Actual Malware) . . . 7
3.3 Triage Sandbox Analysis . . . 8
3.3.1 First Run (Passive Analysis): Score 4/10 . . . 8
3.3.2 Second Run (Interactive Analysis): Score 8/10 . . . 8
3.4 Yaraify Analysis . . . 8

4 Technical Anomalies in the Modified Binary **8**
4.1 RSA Key Size Downgrade . . . 8
4.2 Future Timestamp (Year 2097) . . . 9
4.3 Imphash Discrepancy . . . 9
4.4 Additional Signature Metadata . . . 9

5 Behavioral Analysis: MITRE ATT&CK Mapping **9**
5.1 Session Cookie Theft (T1539) . . . 10
5.2 Runtime Broker Impersonation (T1036.005) . . . 10
5.3 Sandbox Escape (T1497) . . . 10

6 The Zero-Click Delivery Vector **10**
6.1 Attack Surface Analysis . . . 10
6.2 Implications . . . 10

7 Implications for Global Security **11**
7.1 Code Signing Trust Collapse . . . 11
7.2 Affected Systems . . . 11
7.3 Historical Precedents . . . 11

8 Hypothesis Evaluation **11**
8.1 Hypothesis 1: SHA-256 Collision Attack 11
8.2 Hypothesis 2: Microsoft Key Compromise 12
8.3 Hypothesis 3: Signature Verification Bypass 12
8.4 Hypothesis 4: Nation-State Capability 12

9 Repository Presence **13**

10 Recommendations **13**
10.1 Immediate Actions 13
10.2 For Organizations 13
10.3 For Researchers 13

11 Conclusion **13**

Acknowledgments **14**

A Indicators of Compromise (IOCs) **14**

B Analysis URLs **14**

C Verification Instructions **15**

D Sample Download **15**

1 Introduction: A New Class of Threat

On January 9, 2026, during routine use of WhatsApp Web through Google Chrome, a file named `WhatsApp_Installer.exe` downloaded itself to the system without any user interaction. No clicks. No prompts. No consent. A zero-click delivery.

Initial analysis seemed to indicate a clean file: 0 out of 71 antivirus engines on VirusTotal detected any threat. The file bore valid Microsoft Corporation digital signatures. By all standard security metrics, this file should be trusted.

However, comparison with the official WhatsApp installer revealed a fundamental impossibility: both files had valid Microsoft signatures, but their SHA-256 hashes were completely different.

This paper presents evidence that the cryptographic foundations of digital signatures—specifically the SHA-2 family of hash functions used in code signing—may have been compromised. We designate this threat **"Omega Infinity"** to reflect its potentially unlimited implications for global digital security.

1.1 What This Paper Proves

1. Two binaries with different SHA-256 hashes both carry valid Microsoft signatures
2. The modified binary exhibits malicious behaviors (session theft, sandbox escape)
3. Automated security systems are inverted: marking the malware as clean and the legitimate file as suspicious
4. The modified binary uses RSA-2048 instead of RSA-4096 (cryptographic downgrade)
5. The compilation timestamp is set to year 2097 (evasion technique)

1.2 Supplementary Evidence Package

All claims in this paper can be independently verified using the `proof.rar` archive:

- **Directory: `Normal/`**
  Contains the official WhatsApp Installer downloaded directly from https://www.whatsapp.com
  SHA-256: `be15ebfca142f85aa27a081652412356ec9cac504c144d8b4dea2b2d0a4d17ca`
- **Directory: `Modified but with valid digital signatures and indetectable/`**
  Contains the anomalous binary delivered via zero-click attack
  SHA-256: `1f8c98a24f1dc2e22a18ce4218972ce83b7da4d54142d2ca0caeb05225dbc4a9`

Both binaries can be independently verified to have valid Microsoft digital signatures using Windows' native signature verification or tools like `sigcheck`.

2 The Cryptographic Impossibility

2.1 How Digital Signatures Must Work

The Authenticode digital signature system used by Microsoft operates on the following cryptographic principles:

1. A cryptographic hash (SHA-256) is computed over the file contents
2. This hash is signed using the signer's RSA private key

3. The signature, certificate chain, and timestamp are embedded in the PE file
4. Verification computes the hash and checks it against the decrypted signature

The Critical Property: If even a single bit of the file changes, the SHA-256 hash changes completely (avalanche effect), and signature verification MUST fail.

2.2 What We Observe

Table 1: Cryptographic Properties Comparison

Property	Official Binary	Modified Binary
SHA-256 Hash	`be15ebfca142f85...`	`1f8c98a24f1dc2e...`
Signature Validity	**VALID**	**VALID**
Signer	Microsoft Corporation	Microsoft Corporation
RSA Key Size	4096 bits	2048 bits
Timestamp	Contemporary (2025)	Future (2097-12-25)
Imphash	Standard WhatsApp	`f34d5f2d4577ed6d9ceec516c1f5a744`

Both signatures validate successfully under Windows signature verification.

2.3 The Mathematical Impossibility

For two different files to have valid signatures from the same signer, one of the following must be true:

2.3.1 Scenario A: Hash Collision

The attacker found two different file contents that produce the same SHA-256 hash (or a hash that validates under the same signature). This would represent a complete break of SHA-256's collision resistance.

2.3.2 Scenario B: Key Compromise

The attacker obtained Microsoft's private signing key and can sign arbitrary content. This would mean Microsoft's entire code signing infrastructure is compromised.

2.3.3 Scenario C: CA Compromise

The attacker compromised a Certificate Authority and issued fraudulent Microsoft certificates. This would undermine the entire PKI trust model.

2.3.4 Scenario D: Signature Algorithm Break

The attacker found a way to forge RSA signatures without the private key. This would break RSA, the foundation of most public-key cryptography.

All scenarios represent catastrophic security failures.

3 Evidence Analysis

3.1 VirusTotal Analysis

3.1.1 Official Binary (Clean)

- **URL**: https://www.virustotal.com/gui/file/be15ebfca142f85aa27a081652412356ec9cac504c144
- **SHA-256**: `be15ebfca142f85aa27a081652412356ec9cac504c144d8b4dea2b2d0a4d17ca`
- **Detection**: 0/71 engines
- **Signature**: Valid - Microsoft Corporation

3.1.2 Modified Binary (Malware)

- **URL**: https://www.virustotal.com/gui/file/1f8c98a24f1dc2e22a18ce4218972ce83b7da4d54142d
- **SHA-256**: `1f8c98a24f1dc2e22a18ce4218972ce83b7da4d54142d2ca0caeb05225dbc4a9`
- **Detection**: 0/71 engines
- **Signature**: Valid - Microsoft Corporation
- **MITRE ATT&CK**: T1539 (Steal Web Session Cookie) detected

Key Observation: Both files show 0 detections. Both have valid Microsoft signatures. But they have completely different SHA-256 hashes.

3.2 Hybrid Analysis: The Detection Inversion

This is perhaps the most disturbing finding. When analyzed by Hybrid Analysis:

3.2.1 Official Binary (Should Be Clean)

- **URL**: https://hybrid-analysis.com/sample/be15ebfca142f85aa27a081652412356ec9cac504c144d
- **Threat Score**: **35/100 - SUSPICIOUS**
- **AV Detection**: Marked as clean
- **Verdict**: Suspicious behavior detected

3.2.2 Modified Binary (Actual Malware)

- **URL**: https://hybrid-analysis.com/sample/1f8c98a24f1dc2e22a18ce4218972ce83b7da4d54142d2695f081e76b84f1dfb0c8a91
- **Threat Score**: **Low - NO SPECIFIC THREAT**
- **AV Detection**: Marked as clean
- **Verdict**: Clean

THE PARADOX

The legitimate official installer is flagged as **SUSPICIOUS (35/100)**
The modified malware binary is marked as **CLEAN**

The detection system is completely inverted.

This inversion strongly suggests the malware has been specifically engineered to appear MORE trustworthy to automated analysis systems than the legitimate software it impersonates.

3.3 Triage Sandbox Analysis

The modified binary was submitted to Triage:

- **URL**: https://tria.ge/260109-1anqsaa14c

3.3.1 First Run (Passive Analysis): Score 4/10

Standard installer behavior observed.

3.3.2 Second Run (Interactive Analysis): Score 8/10

Upon interaction with the specimen:

- Mouse input became unresponsive (captured by malware)
- Analysis session was forcibly terminated
- Sandbox escape indicators triggered
- Anti-analysis countermeasures activated

Legitimate installers do not fight back against security analysis.

3.4 Yaraify Analysis

- **URL**: https://yaraify.abuse.ch/scan/results/70682ee9-ee5a-11f0-9df4-42010aa4000b

Key findings:

- **Imphash**: `f34d5f2d4577ed6d9ceec516c1f5a744` (does NOT match WhatsApp)
- **Runtime Broker Impersonation**: Detected
- **PowerShell Obfuscation**: Detected
- **Unpacker**: No matches (custom/novel protection)
- **ClamAV**: No matches (complete evasion)

4 Technical Anomalies in the Modified Binary

4.1 RSA Key Size Downgrade

Property	Official	Modified
RSA Key Size	4096 bits	2048 bits

Microsoft's current code signing practices use RSA-4096. The modified binary uses RSA-2048. This downgrade suggests:

- The signature was not created through Microsoft's normal signing infrastructure
- Weaker keys may have been easier to compromise or forge
- This is an intentional evasion of detection based on key strength

4.2 Future Timestamp (Year 2097)

The PE compilation timestamp in the modified binary is:

`2097-12-25 00:56:56 UTC`

This is 71 years in the future. This technique:

- Evades date-range-based security filters
- May cause integer overflow in poorly implemented parsers
- Confuses temporal analysis and correlation
- Is a known APT-level evasion technique

4.3 Imphash Discrepancy

The Import Hash (Imphash) provides a fingerprint of a PE file's imported functions. The modified binary's Imphash:

`f34d5f2d4577ed6d9ceec516c1f5a744`

This does NOT match any known legitimate WhatsApp binary. The import table has been modified, indicating:

- Injection of additional functionality
- Replacement of legitimate API calls
- Addition of malicious imports

4.4 Additional Signature Metadata

The modified binary contains additional metadata in its digital signature that the official binary lacks. This extra data could serve as:

- Covert communication channel
- Configuration data for the malware
- Obfuscation mechanism

5 Behavioral Analysis: MITRE ATT&CK Mapping

Despite 0/71 antivirus detections, behavioral analysis reveals the following MITRE ATT&CK techniques:

Table 2: MITRE ATT&CK Techniques Observed

ID	Technique	Evidence
T1539	Steal Web Session Cookie	File access patterns consistent with browser credential extraction
T1036.005	Masquerading: Match Legitimate Name	Impersonates RuntimeBroker.exe
T1027	Obfuscated Files or Information	PowerShell obfuscation detected
T1497	Virtualization/Sandbox Evasion	Active sandbox escape during Triage analysis
T1189	Drive-by Compromise	Zero-click delivery via WhatsApp Web

5.1 Session Cookie Theft (T1539)

The behavioral analysis identified file system access patterns targeting browser cookie storage. This enables:

- Session hijacking without credentials
- Bypass of multi-factor authentication
- Persistent access to user accounts

5.2 Runtime Broker Impersonation (T1036.005)

RuntimeBroker.exe is a legitimate Windows system process. The malware attempts to masquerade as this process to:

- Evade process-based detection
- Inherit trust associated with system processes
- Hide among legitimate system activity

5.3 Sandbox Escape (T1497)

During Triage analysis, the specimen demonstrated active anti-analysis behavior:

- Detected the sandbox environment
- Disabled mouse input to prevent interaction
- Forcibly terminated the analysis session
- Escalated from 4/10 to 8/10 threat score upon interaction

Legitimate software does not implement anti-forensics measures.

6 The Zero-Click Delivery Vector

The specimen was delivered without any user interaction while accessing WhatsApp Web. This zero-click delivery represents a severe vulnerability:

6.1 Attack Surface Analysis

1. **Browser Exploit**: Chrome vulnerability enabling silent downloads
2. **WhatsApp Web Exploit**: Application-level vulnerability in the web client
3. **Network Injection**: Man-in-the-middle attack injecting the download
4. **CDN Compromise**: Compromise of WhatsApp/Microsoft content delivery

6.2 Implications

Zero-click attacks are the most dangerous class of exploits because:

- No user action required
- No security warnings displayed
- No opportunity for user to prevent infection
- Scales to all users of the affected platform

7 Implications for Global Security

7.1 Code Signing Trust Collapse

If attackers can produce validly signed binaries that differ from legitimate software, the entire code signing trust model fails:

- **Windows SmartScreen**: Trusts Microsoft-signed binaries
- **Antivirus Whitelisting**: Many AVs whitelist signed Microsoft code
- **Enterprise Application Control**: Policies trust code signing
- **Operating System Trust**: Windows inherently trusts Microsoft signatures

7.2 Affected Systems

Every system that relies on SHA-2 based digital signatures:

- All Windows operating systems
- Code signing infrastructure worldwide
- TLS/SSL certificates
- Document signing systems
- Cryptocurrency systems using SHA-256
- Blockchain technologies

7.3 Historical Precedents

Previous code signing compromises have had severe impacts:

- **Flame (2012)**: Used compromised Microsoft certificate
- **Stuxnet (2010)**: Used stolen Realtek and JMicron certificates
- **SolarWinds (2020)**: Legitimately signed malicious updates
- **ASUS Live Update (2019)**: Signed with compromised ASUS keys

Omega Infinity potentially exceeds all of these in severity if it represents a cryptographic break rather than key theft.

8 Hypothesis Evaluation

8.1 Hypothesis 1: SHA-256 Collision Attack

Evidence For:

- Two different files, both with valid signatures
- Would explain the “impossible” observation

Evidence Against:

- No public SHA-256 collision has been demonstrated

- Would require approximately 2^{128} operations
- Would be a historic cryptographic breakthrough

Assessment: If true, this is the most severe scenario. SHA-256 underlies Bitcoin, TLS, code signing, and countless security systems.

8.2 Hypothesis 2: Microsoft Key Compromise

Evidence For:

- Would explain valid Microsoft signatures on modified code
- Precedent exists (Flame malware)

Evidence Against:

- Microsoft's signing infrastructure is highly protected
- Would require insider access or severe breach
- RSA-2048 vs RSA-4096 difference suggests different key

Assessment: Serious but potentially containable through key revocation.

8.3 Hypothesis 3: Signature Verification Bypass

Evidence For:

- Implementation bugs have been found before
- Could explain verification success without cryptographic break

Evidence Against:

- Windows signature verification is extensively tested
- Multiple independent verifiers would need the same flaw

Assessment: Would be severe but patchable.

8.4 Hypothesis 4: Nation-State Capability

Evidence For:

- Sophistication level suggests significant resources
- Zero-click + valid signatures + sandbox escape = APT tradecraft
- Nation-states may have unpublished cryptanalytic capabilities

Evidence Against:

- Would expect more targeted deployment
- Exposure through public analysis is unusual for nation-state tools

Assessment: Most likely actor type given capability requirements.

9 Repository Presence

The specimen has been indexed by major malware repositories:

- **MWDB (CERT.PL)**: https://mwdb.cert.pl/file/1f8c98a24f1dc2e22a18ce4218972ce83b7da4d54
- **VirusShare**: https://virusshare.com/file?1f8c98a24f1dc2e22a18ce4218972ce83b7da4d54142d2c
- **VX-Underground**: Available at https://virus.exchange/samples/
- **Yaraify**: https://yaraify.abuse.ch/scan/results/70682ee9-ee5a-11f0-9df4-42010aa4000b

The security community has recognized this specimen as anomalous, evidenced by its presence in these research repositories despite 0/71 antivirus detections.

10 Recommendations

10.1 Immediate Actions

1. **Microsoft**: Investigate signing infrastructure for compromise
2. **Certificate Authorities**: Audit issuance logs for anomalies
3. **NIST/NSA**: Evaluate SHA-256 status given this evidence
4. **Security Vendors**: Do not whitelist based on signature alone

10.2 For Organizations

1. Implement behavioral analysis regardless of code signing status
2. Monitor for the specific IOCs provided in this paper
3. Do not rely solely on signature validity for trust decisions
4. Implement defense-in-depth assuming signature trust may fail

10.3 For Researchers

1. Verify the claims in this paper using the provided evidence
2. Conduct deep reverse engineering of the modified binary
3. Analyze the exact cryptographic properties of both signatures
4. Investigate the delivery mechanism through controlled reproduction

11 Conclusion

This paper presents evidence of what should be cryptographically impossible: two different binaries, with different SHA-256 hashes, both carrying valid Microsoft Corporation digital signatures.

The implications are severe:

1. Either SHA-2 has been broken (catastrophic for global cryptography)
2. Or Microsoft's signing infrastructure is compromised (catastrophic for Windows security)

3. Or the PKI trust model has a fundamental flaw (catastrophic for internet security)

The evidence is provided for independent verification. The supplementary archive `proof.rar` contains both binaries. The hashes are documented. The analysis URLs are provided. This can be verified by any security researcher with appropriate tools and isolated environments.

We designate this threat **Omega Infinity** because its implications are potentially unlimited. If the cryptographic foundations of digital trust have been compromised, no system relying on digital signatures can be considered secure.

This is not a theoretical concern. The malware was delivered. The signatures verify. The detection systems are inverted. The evidence is real.

The security community must investigate immediately.

Acknowledgments

The author acknowledges the various online security analysis platforms that enabled this research: VirusTotal, Hybrid Analysis, Triage, Yaraify, and the malware research repositories that have indexed this specimen.

A Indicators of Compromise (IOCs)

Table 3: Complete IOC Table

Indicator	Value
Modified Binary (Malware)	
SHA-256	`1f8c98a24f1dc2e22a18ce4218972ce83b7da4d54142d2ca0caeb05225dbc4a9`
Imphash	`f34d5f2d4577ed6d9ceec516c1f5a744`
Filename	`WhatsApp_Installer.exe`
Compilation Time	2097-12-25 00:56:56 UTC
RSA Key Size	2048 bits
Official Binary (Reference)	
SHA-256	`be15ebfca142f85aa27a081652412356ec9cac504c144d8b4dea2b2d0a4d17ca`
RSA Key Size	4096 bits

B Analysis URLs

- **VirusTotal (Official)**: https://www.virustotal.com/gui/file/be15ebfca142f85aa27a0816524123
- **VirusTotal (Modified)**: https://www.virustotal.com/gui/file/1f8c98a24f1dc2e22a18ce4218972
- **Hybrid Analysis (Official)**: https://hybrid-analysis.com/sample/be15ebfca142f85aa27a081652
- **Hybrid Analysis (Modified)**: https://hybrid-analysis.com/sample/1f8c98a24f1dc2e22a18ce42
  695f081e76b84f1dfb0c8a91
- **Triage**: https://tria.ge/260109-1anqsaa14c
- **Yaraify**: https://yaraify.abuse.ch/scan/results/70682ee9-ee5a-11f0-9df4-42010aa4000b

C Verification Instructions

To verify the core claim of this paper:

1. Download both binaries from `proof.rar`
2. Compute SHA-256 hash of both files (should differ)
3. Verify digital signatures using Windows: `Right-click → Properties → Digital Signatures`
4. Or use Sysinternals `sigcheck`: `sigcheck.exe -h <filename>`
5. Both should show valid Microsoft Corporation signatures
6. This result should be impossible

D Sample Download

The modified binary can be downloaded for analysis from:

https://yaraify.abuse.ch/scan/results/70682ee9-ee5a-11f0-9df4-42010aa4000b

WARNING: Handle only in isolated analysis environments. Do not execute on production systems.